# Unit Testing with JUnit

## Concepts

Haluk O. Bingol

Faculty of Computer and Information Sciences
Yeditepe University

v2026-10-18

# Outline

1. Why Automated Tests?
2. Kinds of Tests
3. Anatomy of a Unit Test
4. JUnit
5. Outcomes and Assertions
6. Good Tests
7. Test-Driven Development and Coverage
8. Summary

# Why Automated Tests?

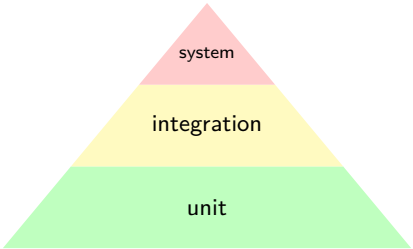
# A bug costs more the later it is found




- A bug found by a test minutes after typing costs minutes; found by a user, it costs days and trust.
- **Regression**: a change breaks something that used to work. Only re-running all tests catches it.
- Tests are also **documentation**: they show how the code is meant to be used.
- They make **change** safe: refactor without fear.

# Kinds of Tests

# From a method to the whole system




| level | tests | speed |
|---|---|---|
| unit | one class or method | ms |
| integration | classes + database, files | seconds |
| system | the whole program | minutes |
| acceptance | what the customer asked for | minutes |

The **pyramid**: many fast unit tests at the bottom, few slow system tests at the top. JUnit is mostly used for unit and integration tests.

# Anatomy of a Unit Test

# Arrange, act, assert

codeoutput/Taste/PriceTest.java — Java

```java
import static org.junit.jupiter.api.Assertions.
    assertEquals;

import org.junit.jupiter.api.Test;

class PriceTest {
    @Test
    void addsTwentyPercent() {
        assertEquals(120, Price.withVat(100));
    }

    @Test
    void zeroStaysZero() {
        assertEquals(0, Price.withVat(0));
    }
}
```

- **Test method**: `@Test`, no parameters, no return value.
- **Arrange**: prepare the input (here: 100).
- **Act**: call the code under test (`withVat`).
- **Assert**: compare with the **expected** value (120); stop with a message if different.

# Arrange, act, assert

codeoutput/Taste/PriceTest.java — Java

```java
import static org.junit.jupiter.api.Assertions.
    assertEquals;

import org.junit.jupiter.api.Test;

class PriceTest {
    @Test
    void addsTwentyPercent() {
        assertEquals(120, Price.withVat(100));
    }

    @Test
    void zeroStaysZero() {
        assertEquals(0, Price.withVat(0));
    }
}
```

- **Test method**: `@Test`, no parameters, no return value.
- **Arrange**: prepare the input (here: 100).
- **Act**: call the code under test (`withVat`).
- **Assert**: compare with the **expected** value (120); stop with a message if different.

Terminal — codeoutput/Taste.txt

```
$ javac -d bin -cp $JUNIT Price.java PriceTest.java
$ junit -cp bin --select-class PriceTest | head -6
.
'-- JUnit Jupiter [OK]
  '-- PriceTest [OK]
    +-- zeroStaysZero() [OK]
    '-- addsTwentyPercent() [OK]
```

# Words of the trade

| term | meaning |
|---|---|
| code under test | the class or method being checked |
| test case | one scenario: input, action, expected result |
| assertion | a check: *expected* versus *actual* |
| fixture | the objects and data a test starts from |
| test suite | a set of tests run together |
| test runner | the program that finds, runs and reports tests |
| test double | a stand-in for a collaborator: stub, fake, mock, spy |

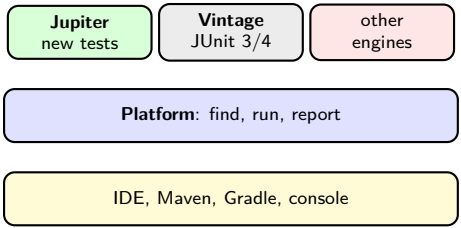
# JUnit

# JUnit: history and parts

JUnit Home: junit.org [1]

| year | version | new |
|---|---|---|
| 1997 | JUnit 1–3 | Kent Beck and Erich Gamma |
| 2006 | JUnit 4 | annotations: `@Test` |
| 2017 | JUnit 5 | Platform + Jupiter, lambdas |
| 2025 | JUnit 6 | Java 17, one version number |

- Free, open source; the standard for Java. Current: **6.1.3**.
- The idea spread to every language: pytest, NUnit, Jest, ... ("xUnit").




Tools speak to the Platform; **engines** run a kind of test. Our tests are **Jupiter** tests.

# How a test class is run

**Algorithm 1:** Running one test class (Jupiter)

**Input:** a class $C$ with test methods $t_1, \ldots, t_n$

1 run the `@BeforeAll` methods
2 **foreach** *test method* $t_i$ **do**
3  create a **new** object of $C$
4  run the `@BeforeEach` methods
5  run $t_i$; record passed / failed / aborted
6  run the `@AfterEach` methods
7 run the `@AfterAll` methods

- A new object per test: tests cannot leak state through fields.
- `@BeforeAll`: expensive setup once (e.g. start a server).
- The order of $t_1, \ldots, t_n$ is fixed but deliberately not obvious: tests must not depend on it.

# Outcomes and Assertions

# Four outcomes

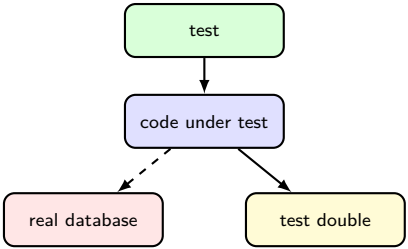
| outcome | IDE | cause |
|---|---|---|
| passed | green | the method ended normally |
| failed | red | an assertion failed, or any other exception was thrown |
| aborted | yellow / gray | an **assumption** did not hold (e.g. "only on **Windows**") |
| skipped | gray | `@Disabled` or a condition, with a reason |

- An **assertion** checks the result: `assertEquals`, `assertTrue`, `assertThrows`, ...
- An **assumption** checks the environment: if it does not hold, the test is not meaningful here.
- "0 tests found" is not success: something is wrong with the setup.

# Good Tests

# FIRST and test doubles

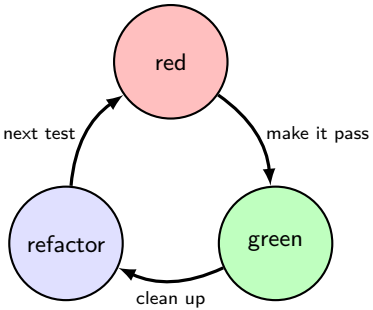
| | |
|---|---|
| **F**ast | run them all the time |
| **I**ndependent | any order, any subset |
| **R**epeatable | same result anywhere |
| **S**elf-validating | pass or fail by itself |
| **T**imely | written with the code |




Slow or unpredictable collaborators (database, network, clock, e-mail) are replaced by **test doubles**.

# Test-Driven Development and Coverage

# Red, green, refactor




1. **Red**: write a test for the next small behavior; see it fail.
2. **Green**: write the simplest code that passes.
3. **Refactor**: improve the code; the tests stay green.

Tests written first are written from the *requirement*, not from the code.

# How good are the tests?

## Coverage

Which lines and branches ran during the tests (tool: JaCoCo). Low coverage shows untested code. High coverage does **not** show good tests: a test without assertions covers code too.

## Mutation testing

A tool makes small bugs on purpose (>= becomes >) and runs the tests. A good test suite *kills* each mutant by failing (tool: PIT).

# Summary

# Big ideas

- Automated tests find bugs early, catch regressions, document behavior and make change safe.
- A unit test: arrange, act, assert; one behavior; fast and independent.
- JUnit: the Platform runs engines; Jupiter is the engine for JUnit 5 and 6 tests.
- A new object per test; `@BeforeEach` prepares the fixture.
- Outcomes: passed, failed, aborted, skipped.
- TDD: red, green, refactor. Coverage finds gaps; mutation testing measures strength.

# Questions to think about

1. Why is “0 tests found, 0 failed” not a good result?
2. Why does JUnit create a new test object for every test method?
3. Give an example of a test that is not *repeatable*. How would you fix it?
4. A method has 100% coverage but a bug. How is that possible?
5. Which is easier to test: a method that prints its result or one that returns it? Why?

# References I

[1] Junit. [Online]. Available: https://junit.org/

# Questions?